Сравнение GCC и IBM XL в STREAM

Команды, параметры сборки и результаты измерений

Условия	Значение
Реализация	Собственная программа polus/stream.cpp; OpenMP.
Вычислительный узел	«Полюс» МГУ, POWER8; 2 сокета по 10 физических ядер; SMT до 8 потоков на ядро.
Компиляторы	GCC 9.3.1; IBM XL C/C++ 13.1.6, C++-драйвер xlC_r.
Основные режимы	GCC -O3; IBM XL -O4; явная настройка под POWER8.
Объём массивов в примере	N = 67 108 864 double; 512 МиБ на массив; три массива — 1,5 ГиБ.
Режим примера	20 физических ядер, 20 потоков, распределение 10+10; local.
Повторяемость	Три независимых запуска для каждой ячейки RAM-примера.
База результатов	polus/data/measurements.csv; polus/data/counters.csv; polus/data/summary/. Сохранённая серия заданий №1614372–1614377.
Дата редакции	13 ноября 2026 года.

Измеряемые величины

Источник	Данные	Результат обработки
stream.cpp	Время четырёх операций; логический объём обработки	ГБ/с для Copy, Scale, Add, Triad.
perf stat	Аппаратные события и события ОС во время выполнения	События/ГБ; IPC; доли ошибок переходов и промахов кэша.
Компилятор; objdump	Отчёты оптимизаций; ассемблер готовой программы	Векторизация, разворачивание, FMA, структура циклов.
plotting/plot.py	Группировка независимых запусков	Медиана, минимум, максимум; графики PNG.

02 / Уровни оптимизации O1–O5

Уровень	GCC 9.3	Классический IBM XL C/C++
-O0	Большинство оптимизаций отключено.	Большинство оптимизаций отключено.
-O1	Базовые преобразования: распространение констант, удаление лишнего кода, упрощение переходов; ограниченные затраты на анализ.	Отдельной ступени -O1 в рассматриваемой линейке XL нет. Основная шкала: 0, 2, 3, 4, 5.
-O2	Дополнительно: глобальное устранение общих выражений, планирование инструкций, оптимизация вызовов и размещения кода.	Основной оптимизированный режим: устранение лишних вычислений, перенос неизменных выражений, оптимизация переходов и инструкций.
-O3	Дополнительно: автовекторизация циклов и блоков; расширенное встраивание; перестановка и другие преобразования циклов. Примеры: -ftree-loop-vectorize, -ftree-slp-vectorize, -finline-functions.	Более затратный анализ и преобразования циклов; -qhot=level=0. По умолчанию допускаются преобразования NOSTRICT, способные менять особенности арифметики.
-O4	Самостоятельной новой ступени нет. Число 4 принимается обработчиком GCC, но не добавляет отдельного стандартного набора оптимизаций сверх -O3.	Дополнительно к -O3: -qhot и межпроцедурная оптимизация -qipa=level=1; автоматическая настройка архитектуры и кэша. Явные параметры после -O4 переопределяют соответствующие настройки.
-O5	Самостоятельной новой ступени нет; отдельного стандартного набора сверх -O3 не добавляет.	Дополнительно к -O4: -qipa=level=2; более полный межпроцедурный анализ потоков данных и возможного пересечения областей памяти.

Термины и дополнительные режимы

Термин / режим	Содержание
Встраивание, inlining	Замена вызова функции её телом; сокращение накладных расходов и расширение области оптимизации.
Векторизация, SIMD	Одна машинная инструкция обрабатывает несколько элементов данных.
IPA	Interprocedural Analysis: анализ через границы функций и единиц компиляции; у XL -O4/-O5 затрагивает также компоновку.
HOT; SLP	High-Order Transformations: преобразования циклов. Superword-Level Parallelism: объединение однотипных скалярных операций в векторные.
GCC -Os; -Og; -Ofast	Оптимизация размера; оптимизация с учётом отладки; -O3 с дополнительными допущениями, включая -ffast-math.
Зависимость от версии	Таблица относится к GCC 9 и классическому XL; IBM Open XL имеет другую шкалу. Источники [1–4].

03 / Выбор GCC -O3 и IBM XL -O4

Основание	GCC -O3	IBM XL -O4
Характер STREAM	Последовательные проходы по массивам; простые арифметические операции. Векторизация снижает число инструкций на элемент.	Та же нагрузка; преобразования циклов и векторная обработка соответствуют структуре программы.
Включённые возможности	В GCC 9 -O3 включает автовекторизацию циклов и SLP; -O2 без дополнительных флагов не включает этот стандартный набор.	-O4 включает HOT и IPA уровня 1; функции ядер и вызывающий OpenMP-код анализируются в более широком контексте.
Выбор относительно младших уровней	-O1/-O2 исключены из основного сравнения, чтобы не ограничивать доступные оптимизации массива базовым режимом.	-O2 сохраняет более строгие условия; -O3 использует менее широкий стандартный набор HOT/IPA, чем -O4.
Выбор относительно старших уровней	-O4/-O5 не являются следующими самостоятельными ступенями. -Ofast меняет дополнительные арифметические допущения.	-O5 усиливает IPA, но преимущество перед -O4 на данном STREAM отдельно не измерялось. -O4 — выбранный базовый режим, а не доказанный максимум.
Контроль архитектуры	-mcpu=power8 -mtune=power8.	-qarch=pwr8 -qtune=pwr8 после -O4.

Область сравнения

Положение	Факт
Предмет	Сопоставляются два практически ориентированных варианта сборки под один POWER8. Номера уровней не образуют общую шкалу между компиляторами.
Межпроцедурная оптимизация	XL -O4 включает IPA на этапе компоновки. У GCC -O3 есть межпроцедурные преобразования, но LTO требует отдельного -flto; в рассматриваемой команде его нет.
Арифметика	У XL высокие уровни допускают NOSTRICT. GCC -O3 без -ffast-math не задаёт тот же набор допущений. Побитовая эквивалентность результатов из этих команд не следует.
Критерий результата	Основной показатель — скорость операций, ГБ/с. События perf и ASM используются для анализа причин различий.
Статус выбора	Режимы -O3/-O4 включают нужные преобразования. Отдельная серия сравнения всех уровней оптимизации не проводилась; максимум производительности среди уровней не установлен.

04 / Команды сборки и флаги

```
SOURCE="$PWD/stream.cpp"
DIR="$PWD/data/campaigns/manual/compilation"
mkdir -p "$DIR"
source lsf/enable_gcc.sh

g++ -O3 -fopenmp -std=c++11 -mcpu=power8 -mtune=power8 \
    -mcmodel=medium "$SOURCE" -o "$DIR/stream"

xlC_r -O4 -q64 -qarch=pwr8 -qtune=pwr8 -qsmp=omp \
      -qlanglvl=extended0x -qmaxmem=-1 "$SOURCE" -o "$DIR/stream"
```

Параметр	Назначение
SOURCE; DIR	Абсолютный путь к исходнику; каталог результата. Команды GCC и XL — альтернативные сборки; имя stream в примере совпадает.
source lsf/enable_gcc.sh	Подключение подготовленного GCC через PATH и LD_LIBRARY_PATH.
-fopenmp / -qsmp=omp	Обработка OpenMP; подключение среды многопоточного выполнения.
-std=c++11 / -qlanglvl=extended0x	Выбор C++11; у XL — режим с расширениями.
-mcpu=power8 / -qarch=pwr8	Доступный набор инструкций POWER8.
-mtune=power8 / -qtune=pwr8	Настройка генерации и планирования инструкций под POWER8.
-mcmodel=medium	Модель адресации статических данных PowerPC64; не размер динамических массивов.
-q64	64-битный режим; не число потоков и не ширина вектора.
-qmaxmem=-1	Снятие лимита памяти для отдельных затратных оптимизаций компилятора; не лимит памяти программы.
-o "$DIR/stream"	Путь к создаваемому исполняемому файлу.
N, threads, CPU	Параметры запуска, а не компиляции. Один бинарник внутри задания используется для разных размеров N.
Архивная сборка	Содержала также -DSTREAM_RUN_INFO для диагностики привязки. Текущий упрощённый исходник этот макрос не использует.

05 / Отчёты компилятора и ASM

```
g++ -O3 -fopenmp -std=c++11 -mcpu=power8 -mtune=power8 \
    -mcmodel=medium -S -fverbose-asm \
    -fopt-info-vec-all="$DIR/vectorization.log" \
    "$SOURCE" -o "$DIR/compiler.s"

cd "$DIR"
xlC_r -O4 -q64 -qarch=pwr8 -qtune=pwr8 -qsmp=omp \
      -qlanglvl=extended0x -qmaxmem=-1 -S -qlist -qreport \
      "$SOURCE" -o compiler.s

objdump -d -C "$DIR/stream" > "$DIR/disassembly.log"
g++ --version > "$DIR/gcc_version.log"
xlC_r -qversion > "$DIR/xl_version.log" 2>&1
```

Флаг / инструмент	Выход и содержание
-S	Ассемблер compiler.s вместо исполняемого файла; отдельный отчётный проход компиляции.
-fverbose-asm	Комментарии к ассемблеру GCC.
-fopt-info-vec-all=…	vectorization.log: успешная векторизация, пропущенные случаи, пояснения.
-qlist; -qreport	Листинг XL и сведения о преобразованиях; обычно .lst. Отдельного листинга XL в текущем архиве нет.
objdump -d	Дизассемблирование секций кода готовой программы.
objdump -C	Читаемые C++-имена функций.
disassembly.log / linked.asm	Текст готового бинарника: актуальное имя в run.py / имя в сохранённой серии.
compiler.s и linked.asm	Могут отличаться из-за этапа компоновки и IPA. Для анализа измерявшейся программы основным является linked.asm.
--version / -qversion	Версия компилятора. В run.py результат выбранной команды сохраняется как version.log.
Число строк ASM	Статический объём текста; не число динамически выполненных инструкций.
stream.elf	Сохранённая исполняемая программа Linux/POWER8; двоичный файл. Источник для повторного дизассемблирования; текстовая версия уже сохранена как linked.asm.

06 / Запуск и события perf

```
OUT="$DIR/../raw"
mkdir -p "$OUT"
CORE=task-clock,cycles,instructions,branches,branch-misses
CORE=$CORE,context-switches,cpu-migrations,page-faults

# plain
LC_ALL=C numactl --localalloc "$DIR/stream" 67108864 2 7 1 0,80 \
  > "$OUT/plain.csv" 2> "$OUT/plain.stderr.log"

# core
LC_ALL=C perf stat -x ";" -o "$OUT/core.perf.csv" -e "$CORE" \
  -- numactl --localalloc "$DIR/stream" 67108864 2 7 1 0,80 \
  > "$OUT/core.csv" 2> "$OUT/core.stderr.log"

# cache
LC_ALL=C perf stat -x ";" -o "$OUT/cache.perf.csv" \
  -e task-clock,cycles,cache-references,cache-misses \
  -- numactl --localalloc "$DIR/stream" 67108864 2 7 1 0,80 \
  > "$OUT/cache.csv" 2> "$OUT/cache.stderr.log"
```

Элемент	Значение
plain / core / cache	Три отдельных запуска при одинаковой конфигурации. Скорость для графиков — из plain; основные счётчики — из core; кэш — из cache.
perf stat; -e	Суммарный счёт событий выполнения; выбор событий. Счётчики агрегируются по отслеживаемым потокам.
-x ";"; -o	Разделитель полей ; и отдельный файл вывода perf.
--	Конец параметров perf; далее — измеряемая команда.
>; 2>	stdout STREAM в CSV; stderr в диагностический файл.
67108864 2 7 1 0,80	N; 2 потока; минимум 7 отсчётов на операцию; один повтор в отсчёте; CPU 0 и 80 — по одному ядру на сокет.
LC_ALL=C	Локаль чисел и сообщений; десятичная точка.
Раздельные наборы событий	Уменьшение конкуренции за аппаратные счётчики. Отсутствие мультиплексирования проверяется по running_percent.

07 / Счётчики: термины и единицы

Событие	Единица	Содержание
instructions	Инструкции	Завершённые машинные инструкции. Одна векторная инструкция может обрабатывать несколько чисел; строки C++ и инструкции не совпадают.
cycles	Такты	Такты, учитываемые аппаратным событием для отслеживаемых потоков; включают периоды ожидания. Сумма не равна времени по часам или уникальным тактам физических ядер при SMT.
branches	Переходы	Выполненные инструкции управления потоком; циклы, вызовы и возвраты в пределах покрытия события. Не количество if в исходнике.
branch-misses	Промахи	Ошибки предсказания переходов; не ошибки вычисленного результата. Цена промаха зависит от процессора и ситуации.
cache-references	Обращения	Обобщённое событие обращений к кэшу. В штатном драйвере Linux POWER8 — PM_LD_REF_L1.
cache-misses	Промахи	Обобщённое событие промахов кэша. В штатном драйвере Linux POWER8 — PM_LD_MISS_L1. Промах L1 может обслужить следующий уровень кэша.
task-clock	Обычно мс	Суммарное CPU-время отслеживаемой задачи и её потоков. 20 работающих потоков за 8 с могут накопить около 160 000 мс.
context-switches	События	Смена выполняющегося потока планировщиком ОС; связана с ожиданием, вытеснением, синхронизацией. Не переход внутри цикла.
cpu-migrations	События	Перемещения выполнения между логическими CPU. Не миграция страниц между NUMA-узлами; общий счёт включает этап до привязки потоков.
page-faults	События	Обработка промахов страниц; в том числе первое выделение страниц массивов. Малый промах не требует чтения диска.

Производные показатели

Показатель	Формула / область применения
IPC	instructions / cycles; события одного core-прохода. Больший IPC не задаёт больший ГБ/с.
Ошибки переходов, %	100 × branch-misses / branches; один core-проход.
Промахи кэша, %	100 × cache-misses / cache-references; один cache-проход. Не доля времени ожидания ОЗУ.
Соответствие событий	Назначение generic-событий зависит от CPU и драйвера. Сопоставление именно ядра «Полюса» отдельно не зафиксировано; источник штатного POWER8-драйвера — [9].

08 / Расчёты и выходные данные

Операция	Формула	Логических байт на элемент
Copy	c[i] = a[i]	16: чтение + запись double.
Scale	b[i] = 3 × c[i]	16: чтение + запись double.
Add	c[i] = a[i] + b[i]	24: два чтения + запись.
Triad	a[i] = b[i] + 3 × c[i]	24: два чтения + запись.

```
ГБ/с = bytes_per_element × N × repeats / seconds / 10^9
processed_gb(kernel) = bytes_per_element × N × repeats × samples / 10^9
processed_gb(process) = сумма processed_gb четырёх операций
per_gb(event) = value(event) / processed_gb(process)
млн событий/ГБ = per_gb(event) / 10^6

Пример: instructions = 217 995 131 500; processed_gb = 1 262,183513
217 995 131 500 / 1 262,183513 / 10^6 = 172,712707 млн инструкций/ГБ
```

Поле / правило	Содержание
Логический ГБ	10^9 байт по формулам STREAM. Не прямой трафик шины ОЗУ; загрузка строк при записи и служебный трафик отдельно не учитываются.
repeats; samples; runs	Повторы ядра в отсчёте; число временных отсчётов; число независимых запусков. В run.py RUNS = 3; perf -r не используется.
Условие остановки операции	Не менее 7 отсчётов и не менее 2 с суммарных измерений.
median_gbps; best_gbps	Медианная и максимальная скорость внутренних отсчётов. Для графика используется median_gbps из plain.
Медиана сводки	Для скорости — медиана медиан процессов; для событий — медиана per_gb; для IPC и процентов — медиана отношений, вычисленных по каждому процессу.
minimum; maximum; runs	Минимум, максимум и число независимых запусков. Полосы разброса — min–max; не доверительный интервал.
running_percent	Процент времени активности счётчика; не загрузка CPU. Выданные perf значения уже могут быть масштабированы. В сохранённой базе — 100,0%.
Область perf	Весь процесс: инициализация, прогрев, OpenMP, четыре операции, завершение. processed_gb учитывает только измеряемые отсчёты; счётчики не разделены по операциям.

Выход	Структура
stdout STREAM	Заголовок и четыре строки: Copy, Scale, Add, Triad. Условия, число отсчётов, объём, время и скорость каждой операции.
Файл perf -o	Разделитель ;. Поля: value; unit; event; counter_runtime; running_percent; дополнительные поля. Единица может быть пустой; возможны строки комментариев.

09 / Векторизация в сохранённом ASM

GCC: основной векторный цикл Triad

```
10002e00: lxvd2x    vs0,r11,r10
10002e04: lxvd2x    vs11,r31,r10
10002e08: xvmaddmdp vs0,vs12,vs11
10002e0c: stxvd2x   vs0,r6,r10
10002e10: addi      r10,r10,16
10002e14: cmpld     cr7,r3,r10
10002e18: bne       cr7,10002e00
```

IBM XL: характерные строки развёрнутого цикла Triad

```
10005b08: addi      r7,r7,128
10005b0c: xvmaddadp vs11,vs4,vs0
          ... ещё семь векторных FMA и загрузки/записи ...
10005b8c: addi      r10,r10,128
10005b94: addi      r6,r6,128
10005b9c: bdnz      10005b00
```

Инструкция / свойство	GCC	IBM XL
Векторные загрузки / записи	lxvd2x / stxvd2x	lxvd2x / stxvd2x
Векторная FMA	xvmaddmdp	xvmaddadp
Размер вектора	128 бит; 2 double	128 бит; 2 double
Работа за оборот основного цикла	2 double; 1 векторная FMA	16 double; 8 векторных FMA
Сдвиг указателя / индекса	16 байт	128 байт
Организация переходов	Проверка после 2 элементов	Проверка после 16 элементов
Файл linked.asm	gcc/a3fdb7bb4d59/	xl/6bb6e09eef92/

vectorization.log GCC

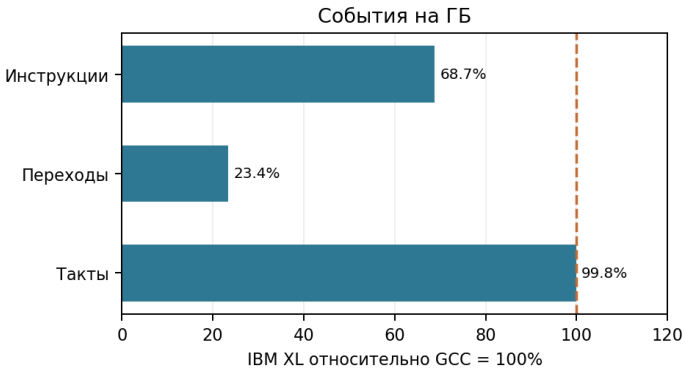
Тип сообщения	Значение
optimized	Оптимизация применена в указанном месте.
missed	Конкретная оптимизация пропущена; сообщение не является ошибкой сборки.
note; statement clobbers memory	Детали анализа; необходимость учитывать возможное влияние операции на память.
Сохранившиеся внутренние циклы	Copy — строка 51; Scale — 59; Add — 68; Triad — 77 текущего stream.cpp.

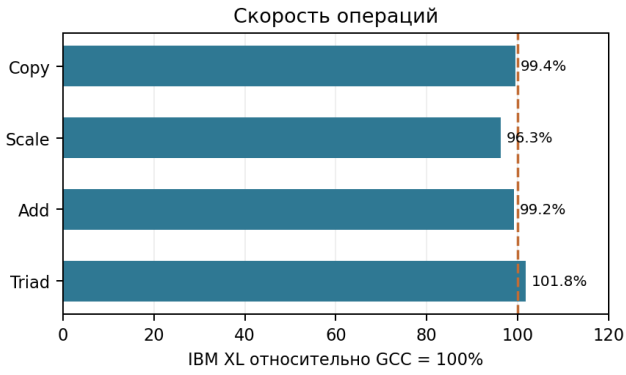
10 / Результаты: GCC и IBM XL

Условия	Значение
Профиль	20 ядер / 20 потоков; 10+10; local; N = 67 108 864.
Сводка	Медианы трёх независимых запусков каждого соответствующего прохода.

Показатель	GCC 9.3.1	IBM XL 13.1.6
Инструкции, млн/ГБ	172,713	118,688
Переходы, млн/ГБ	32,190	7,535
Такты, млн/ГБ	479,270	478,539
Ошибки переходов, млн/ГБ	0,00802	0,14345
IPC, инструкций/такт	0,3599	0,2481
Ошибки переходов, %	0,0248	1,9038
Промахи кэша, %	16,204	17,688

Операция	GCC, ГБ/с	XL, ГБ/с	Изменение XL
Copy	149,453	148,616	-0,56%
Scale	149,381	143,843	-3,71%
Add	181,867	180,418	-0,80%
Triad	181,732	184,973	+1,78%





Наблюдение	Численный результат
Сокращение работы XL	−31,28% инструкций/ГБ; −76,59% переходов/ГБ.
Изменение времени / скорости	Такты/ГБ: около −0,15%; ГБ/с: от −3,71% до +1,78% в зависимости от операции.

11 / NUMA и выводы по измерениям

20 ядер, 10+10	local, ГБ/с	interleave, ГБ/с	Изменение
GCC / Copy	149,45	76,46	-48,8%
GCC / Triad	181,73	87,41	-51,9%
IBM XL / Copy	148,62	72,09	-51,5%
IBM XL / Triad	184,97	88,51	-52,2%

Режим / наблюдение	Факт и интерпретация
numactl --localalloc	Предпочтение памяти узла выполняющегося потока. Параллельное первое касание после привязки обычно размещает страницы рядом с владельцем фрагмента. Это не жёсткая привязка памяти.
numactl --interleave=0,1	Чередование страниц между узлами 0 и 1. Часть обращений к собственному фрагменту становится удалённой. C++-код в двух режимах одинаковый.
Снижение ГБ/с при interleave	В приведённых примерах около 49–52%. Согласуется с влиянием локальности памяти и межсокетных обращений.
Меньше инструкций у XL	Подтверждено perf; согласуется с более сильным разворачиванием основного цикла. Скорость не возрастает пропорционально сокращению команд.
IPC и branch-misses	Более высокий IPC GCC не обеспечивает преимущество во всех операциях. У XL больше ошибок переходов и на ГБ, и в процентах. Причина и цена отдельных промахов суммарным perf не локализованы.
Разброс Scale, ГБ/с	GCC: 149,273–149,455; XL: 142,852–147,122. Диапазоны трёх запусков не пересекаются.
Разброс Triad, ГБ/с	GCC: 181,722–181,979; XL: 184,491–185,113. Диапазоны не пересекаются. У Add пересекаются.
Статистический вывод	Списание всех отличий на шум не подтверждается диапазонами. Три наблюдения и min–max не заменяют оценку статистической значимости.
Малые N	Данные могут обслуживаться кэшем; вклад запуска OpenMP относительно выше. ГБ/с при малом N не тождественен пропускной способности ОЗУ.
20+0 и 0+20	20 потоков на 10 физических ядрах одного сокета, по 2 потока на ядро. В серии SMT число физических ядер зависит от раскладки.
Область сохранённой серии	Включала диагностику run_info и проверку результата. В текущем упрощённом исходнике эти действия удалены; состав накладной работы изменён.

12 / Файлы проекта и источники

Путь относительно polus	Назначение
run.py; stream.cpp; profiles.py	Сборка и обход конфигураций; измеряемая программа; списки CPU.
data/measurements.csv; data/counters.csv	Общая база сохранённых скоростей и событий.
data/campaigns/run_<JOB>/	Новые запуски: raw/*.csv, *.perf.csv, *.stderr.log; объединённые measurements.csv и counters.csv; manifest.json.
data/summary/	speeds.csv — скорость plain; counters.csv — четыре события core в млн/ГБ; metrics.csv — IPC и проценты.
data/compilation/	compiler.s; linked.asm; stream.elf; vectorization.log GCC; снимки исходников source_snapshots.json.
graphs/; plotting/plot.py	Графики PNG; скрипт расчёта сводок и построения.

```
# Локально из корня проекта:
py -3 polus/plotting/plot.py

# На выделенном узле, из polus:
python3 run.py --compiler gcc
python3 run.py --compiler xl
```

Условие запуска	Значение
STREAM_PROFILE_INDEX	При наличии — один профиль на задание LSF; при отсутствии — все профили.
Состояние работы	При подготовке PDF программы на «Полюсе» не запускались; использованы локальные сохранённые данные.

Документация

[1] GCC 9.3: уровни оптимизации

[2] GCC 9.3: обработчик числового -O; таблица стандартных флагов

[3] IBM XL: уровни оптимизации классической линейки

[4] IBM XL Linux 13.1.6: Compiler Reference

[5] IBM XL: IPA и дополнительные настройки O4

[6] perf stat: параметры и формат вывода

[7] perf_event_open: аппаратные и программные события

[8] perf: наследование событий и масштабирование

[9] Linux: power8_generic_events в драйвере POWER8

[10] GCC 9.3: отчёты оптимизатора

[11] GNU Binutils: objdump

[12] numactl: политики памяти